

Frontend Interview Prep Notes

HTML • CSS • JavaScript • React • Browser • System Design

Section 1 — HTML & CSS

Markup Language

A markup language uses **tags and codes** to structure web content. HTML (HyperText Markup Language) is the standard markup language for web pages.

- **Tag:** `< >` — the building block syntax
- **Element:** Combination of opening and closing tag → `<div>hello</div>`
- **Attribute:** Attached to HTML tags to provide additional info → `src`, `class`, `href`, `name`
- **Property:** The value assigned after `=` → `class="container"`
- Attributes **initialize** properties. Once the browser parses HTML, attributes become DOM properties.

DOCTYPE

```
<!DOCTYPE html>
```

Tells the browser which version of HTML the document uses. `<!DOCTYPE html>` means HTML5. The first file that executes is `index.html`.

Inline vs Block vs Inline-Block

Type	Behavior	Examples
Block	Takes full width of parent, always starts on a new line	<code>div</code> , <code>p</code> , <code>h1</code>
Inline	Takes only needed space, ignores width/height	<code>span</code> , <code>button</code> , <code>a</code>
Inline-Block	Sits inline but accepts width, height, padding	<code>img</code>

- **Block con:** Wastes horizontal space
- **Inline:** Won't accept `width` / `height`
- **Inline-Block:** Best of both worlds

CSS Box Model

Every HTML element is a rectangular box with 4 layers (*outside* → *inside*):

Margin → Border → Padding → Content

- **Content:** The actual text/image
- **Padding:** Space between content and border
- **Border:** The outline of the element
- **Margin:** Space outside the border (separation from other elements)

```
.box {
  width: 200px;      /* content width */
  padding: 10px;     /* inner space */
  border: 2px solid; /* outline */
  margin: 20px;      /* outer space */
}
```

Section 2 — JavaScript

let, var, const

	var	let	const
Scope	Function	Block	Block
Hoisted	Yes (as <code>undefined</code>)	Yes (TDZ)	Yes (TDZ)
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No

Hoisting

Hoisting = JS reserves memory for variable and function declarations **before** code runs (during the Memory Creation Phase).

- `var` → hoisted and initialized as `undefined`
- `let` / `const` → hoisted but in **TDZ** (cannot access before declaration)
- **Function statements** → fully hoisted, can be called before declaration

```
console.log(a); // undefined (var hoisted)
var a = 5;

console.log(b); // ReferenceError (TDZ)
let b = 5;

greet(); // Works! Function statements are fully hoisted
function greet() { console.log("Hello"); }
```

Temporal Dead Zone (TDZ)

The TDZ is the area in a block where a variable is **in scope but cannot be accessed** until it is declared and initialized. Accessing it throws a `ReferenceError`.

```
console.log(x); // ReferenceError: Cannot access 'x' before initialization
let x = 10;
```

Execution Context

The **environment** in which JavaScript code is evaluated and executed.

- **GEC (Global Execution Context)**: Created when JS first starts running. Only one GEC exists.

- **FEC (Function Execution Context):** Created every time a function is invoked. Each call gets its own FEC.

Each execution context has two phases:

1. **Memory Creation Phase** → Variables hoisted, functions stored in memory
2. **Code Execution Phase** → Code runs line by line

Scope & Lexical Scoping

- **Local/Function Scope:** Variables inside a function are only accessible within it
- **Block Scope:** `let` / `const` inside `{ }` are block-scoped
- **Lexical Scope:** A function can access variables from its outer scope based on where it was **defined** (not where it's called)

```
function outer() {  
  let x = 10;  
  function inner() {  
    console.log(x); // 10 - lexical scope gives inner access to outer's x  
  }  
  inner();  
}
```

Closures

A **closure** is a function that retains access to its own scope **and** its outer (lexical) scope even after the outer function has finished executing.

```
function counter() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const increment = counter();  
increment(); // 1  
increment(); // 2 - count persists in closure
```

Function Types

Type	Syntax	Notes
Function Statement	<code>function greet() {}</code>	Fully hoisted
Function Expression	<code>const greet = function() {}</code>	Not hoisted
Arrow Function	<code>const greet = () => {}</code>	No own <code>this</code>
IIFE	<code>(function() {})()</code>	Runs immediately, no reference stored
Anonymous	<code>function() {}</code>	No name
Higher Order (HOF)	Takes or returns a function	e.g., <code>map</code> , <code>filter</code>
Callback	Passed as an argument	Called later by another function
First-Class Function	Functions treated as values	Can be assigned, passed, returned

Function Statement vs Expression

```
// Statement - hoisted
function b() { }

// Expression - not hoisted
var a = function b() { };
```

IIFE (Immediately Invoked Function Expression)

Runs immediately and is then discarded — **does not store any reference**, conserves memory space.

```
(function() {
  console.log("Runs once, immediately");
}) ( );
```

this Keyword

`this` refers to the **object currently executing the code**. Its value depends on **how** the function is called, not where it's defined.

Context	this refers to
Global scope	<code>window</code> (browser) / <code>global</code> (Node)
Object method	The object itself
Arrow function	Inherited from outer scope (no own <code>this</code>)
<code>call</code> / <code>apply</code> / <code>bind</code>	Explicitly set

call, apply, bind

All three let you **explicitly set** `this`:

```
const person = { name: "Kush" };

function greet(city, country) {
  console.log(`${this.name} from ${city}, ${country}`);
}

greet.call(person, "Delhi", "India"); // args one by one - invokes immediately
greet.apply(person, ["Delhi", "India"]); // args as array - invokes immediately
const bound = greet.bind(person, "Delhi"); // returns new function - call later
bound("India");
```

Type Coercion vs Type Casting

- **Type Coercion:** JS **automatically** converts types (implicit)
- **Type Casting:** **Manually** converting types (explicit)

```
// Coercion (implicit)
"5" + 2 // "52" - number coerced to string (+ triggers string concat)
"5" - 2 // 3 - string coerced to number

// Casting (explicit)
Number("5") // 5
String(5) // "5"
Boolean(0) // false
```

JS converts all mathematical operators **except** `+` **and** `NaN` to Number before any comparison or

calculation.

Spread vs Rest

Operator	Role	Use
Spread <code>...arr</code>	Expands elements	Copying arrays/objects, passing args
Rest <code>...args</code>	Collects elements	Function parameters

```
const arr = [1, 2, 3];
Math.max(...arr); // Spread: expands to Math.max(1, 2, 3)

function sum(...nums) { // Rest: collects all args into array
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3); // 6
```

Deep Copy vs Shallow Copy

- **Shallow Copy:** Copies only the **top level**. Nested objects are still **referenced** (shared).
- **Deep Copy:** Completely **independent** copy at every level.

```
// Shallow Copy - spread operator
const copy = { ...original };
copy.address.city = "Mumbai"; // also changes original.address.city!

// Deep Copy - JSON method
const deepCopy = JSON.parse(JSON.stringify(original));
deepCopy.address.city = "Mumbai"; // original is untouched
```

Limitation of JSON method: Loses `undefined`, functions, `Date`, `RegExp`.

Modern alternative: `structuredClone(original)` — handles more types correctly.

Pass by Value vs Pass by Reference

- **Primitives** (number, string, boolean) → **by value** (copy is made)
- **Objects/Arrays** → **by reference** (both point to same memory location)

```
let a = 5;
let b = a;
b = 10;
console.log(a); // 5 - unchanged, b got a copy

let obj = { x: 1 };
let obj2 = obj;
obj2.x = 99;
console.log(obj.x); // 99 - same reference
```

Prototype & Prototypal Inheritance

Every object in JS has a **hidden** `[[Prototype]]` property pointing to its parent. This forms the **prototype chain**.

- All objects ultimately end at `null` at the top of the chain.
- Enables **inheritance** — child objects can access methods/properties of their parent.

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  console.log(`${this.name} makes a sound`);
};

const dog = new Animal("Dog");
dog.speak(); // "Dog makes a sound" - inherited from Animal.prototype
```

- `Object.setPrototypeOf(obj, parent)` — sets prototype
- `Object.getPrototypeOf(obj)` — gets prototype

Constructor Function

A regular function used with `new` to create objects with a shared structure.

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const p1 = new Person("Kush", 22);
```


Polyfill

Code that **provides modern functionality on older browsers** that don't natively support it.

Common polyfills to know how to implement: `call`, `bind`, `apply`, `filter`, `map`, `reduce`, `forEach`, `Promise.all`

Currying

Transforms a function taking multiple arguments into a series of functions each taking **one argument at a time**.

```
// Infinite curry example
function add(a) {
  return function(b) {
    if (b !== undefined) return add(a + b);
    return a;
  };
}
add(1)(2)(3)(); // 6
```

Array Methods

slice vs splice

	slice(start, end)	splice(start, deleteCount)
Mutates original?	No	Yes
Returns	New array (extracted portion)	Array of removed elements
Use	Extract / copy	Add or remove elements

```
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 3); // [2, 3] - original unchanged
arr.splice(1, 2); // removes 2 from index 1; arr becomes [1, 4, 5]
```

reduce

Reduces an array to a **single value** by running a function on each element:

```
const sum = [1, 2, 3, 4].reduce((accumulator, current) => accumulator + current, 0);
// 0+1=1 → 1+2=3 → 3+3=6 → 6+4=10
// Result: 10
```

Promises

An object representing the **eventual completion or failure** of an asynchronous operation.

States: `pending` → `fulfilled` or `rejected`

```
// Creating a Promise using constructor
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Done!"), 1000);
  // Or: reject("Error!")
});

// Consuming
p.then(val => console.log(val))
  .catch(err => console.error(err))
  .finally(() => console.log("Always runs"));
```

Promise Chaining

```
fetch('/api/user')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Promise Combinators

Method	Behavior
<code>Promise.all([...])</code>	Waits for all to fulfill. Rejects if any one rejects
<code>Promise.allSettled([...])</code>	Waits for all . Returns status+value of each regardless of outcome
<code>Promise.race([...])</code>	Resolves/rejects as soon as first one settles
<code>Promise.any([...])</code>	Resolves with first fulfilled . Rejects only if all fail

Async / Await

Syntactic sugar over Promises — makes async code look and read synchronously.

```
async function fetchData() {
  try {
    const res = await fetch('/api/data'); // waits for promise to resolve
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

Event Loop

JS is **single-threaded** but handles async via the Event Loop.

Order of execution:

1. **Call Stack** — executes synchronous code
2. **Web APIs** — handles async tasks outside JS (setTimeout, fetch, DOM events)
3. **Microtask Queue** — Promise `.then()` callbacks — **higher priority**
4. **Callback Queue (Macrotask)** — setTimeout/setInterval callbacks
5. **Event Loop** — when Call Stack is empty: processes all microtasks first, then one macrotask

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
console.log("end");

// Output: start → end → promise → timeout
```

Starvation: If microtasks keep queuing more microtasks infinitely, macrotasks (setTimeout) never get to execute.

Web APIs

Browser-provided APIs that are **not part of the JS engine** — they run outside the call stack:

`setTimeout`, `setInterval`, `fetch`, `DOM API`, `localStorage`, `Geolocation`, etc.

When the async task completes, the callback is pushed to the appropriate queue for the Event Loop to pick up.

async and defer (Script Loading)

```
<script src="app.js"></script>           <!-- blocks HTML parsing -->
<script async src="app.js"></script>      <!-- downloads parallel, executes when ready
<script defer src="app.js"></script>      <!-- downloads parallel, executes after HTML
```

	async	defer
Download	Parallel to HTML parsing	Parallel to HTML parsing
Execute	Immediately when downloaded (pauses HTML)	After HTML fully parsed
Order guaranteed	No	Yes (in order)
Use case	Independent scripts (analytics)	Scripts that need the DOM

Cookies, localStorage, sessionStorage

	Cookies	localStorage	sessionStorage
Capacity	~4KB	~5-10MB	~5MB
Expiry	Set manually	Never (until cleared)	When tab/window closes
Sent with HTTP requests	Yes (auto)	No	No
Access	JS + Server	JS only	JS only

Event Delegation, Bubbling, Capturing

- **Event Bubbling:** Event travels from **target** → **up to root** (default behavior)
- **Event Capturing:** Event travels from **root** → **down to target** (use `{capture: true}`)
- **Event Delegation:** Attach **one listener on parent** to handle events from all children — efficient, works for dynamically added elements

```
// Event Delegation Example
document.getElementById("list").addEventListener("click", function(e) {
  if (e.target.tagName === "LI") {
    console.log("Clicked:", e.target.textContent);
  }
});

e.stopPropagation() // Stops event from bubbling/capturing further
e.preventDefault() // Prevents default browser behavior
```

DOM (Document Object Model)

A **tree-structured** representation of the HTML elements in a webpage. Represents the entire UI of your application as a tree of nodes.

```
document
├── html
│   ├── head
│   └── body
│       ├── div
│       │   └── p → "Hello"
│       └── span
```

Errors

- **Syntax Error:** Code violates JS grammar rules (e.g., `let = 5;`)
- **Reference Error:** Accessing an undeclared variable or one in TDZ

Section 3 — React

What is React & Why React?

React is a **JavaScript library** for building user interfaces, made by Meta (Facebook).

Why React?

- **Component-based** → reusable, maintainable UI pieces

- **Virtual DOM** → efficient updates (only re-renders what changed)
- **One-way data flow** → predictable state management
- **Declarative** → describe what UI should look like; React handles the how

Virtual DOM

A **lightweight in-memory representation** of the actual DOM. When state changes:

1. React builds a **new Virtual DOM tree**
2. **Diffing**: Compares new VDOM with the **pre-updated VDOM** (snapshot before change)
3. **Reconciliation**: Only the changed nodes get updated in the real DOM

This avoids expensive full-page re-renders.

Reconciliation & React Fiber

Reconciliation: The process React uses to update the actual DOM to match the virtual DOM — identifying what changed and updating only those parts efficiently.

React Fiber (React 16+): The reconciliation engine rewrite that enables **incremental rendering** — breaking render work into chunks, allowing React to pause, prioritize, and resume work across frames (smoother animations and interactions).

Component Lifecycle

Phase	Class Method	Hooks Equivalent
Mounting	<code>componentDidMount</code>	<code>useEffect(() => {}, [])</code>
Updating	<code>componentDidUpdate</code>	<code>useEffect(() => {}, [dep])</code>
Unmounting	<code>componentWillUnmount</code>	Cleanup function in <code>useEffect</code>

```
// Runs once on mount (componentDidMount)
useEffect(() => {}, []);

// Runs when 'dep' changes (componentDidUpdate)
useEffect(() => {}, [dep]);

// Cleanup on unmount (componentWillUnmount)
useEffect(() => {
  return () => { /* cleanup: clear interval, unsubscribe */ };
}, []);
```

Props vs State

	Props	State
Passed from	Parent to child	Managed within component
Mutable	No (read-only)	Yes
Triggers re-render	On parent re-render	On state change via setter

```
// Props
function Child({ name }) { return <div>{name}</div>; }

// State
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Props Drilling

Passing data through **many nested component levels** even when intermediate components don't need it.

- **Problem:** Tight coupling, hard to maintain
- **Solution:** Context API, Redux, or Zustand

Synthetic Events

React wraps native browser events in **SyntheticEvent** — a cross-browser wrapper ensuring consistent event behavior across all browsers.

```
function handleClick(e) {  
  e.preventDefault(); // consistent behavior in all browsers  
  e.stopPropagation();  
}
```

useState

```
const [state, setState] = useState(initialValue);
```

Manages **local component state**. Every `setState` call triggers a re-render.

useReducer vs useState

- **useState**: For simple, independent state (counter, toggle, input)
- **useReducer**: For complex state logic where next state depends on previous, or multiple sub-values

```
function reducer(state, action) {  
  switch (action.type) {  
    case "INCREMENT": return { count: state.count + 1 };  
    case "RESET":      return { count: 0 };  
    default:           return state;  
  }  
}  
  
const [state, dispatch] = useReducer(reducer, { count: 0 });  
dispatch({ type: "INCREMENT" });
```

Custom Hook

A function starting with `use` that can call other hooks. Used to **extract and reuse stateful logic** across components.


```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(url)
      .then(r => r.json())
      .then(d => { setData(d); setLoading(false); });
  }, [url]);

  return { data, loading };
}

// Usage in component
const { data, loading } = useFetch('/api/users');
```

useMemo vs useCallback

	useMemo	useCallback
Returns	Memoized value	Memoized function
Recomputes when	Dependency changes	Dependency changes
Use for	Expensive calculations	Stable function reference for child components

```
const result = useMemo(() => expensiveCalc(data), [data]);
const handler = useCallback(() => doSomething(id), [id]);
```

Memoization & React.memo

Memoization: Caching results of expensive function calls so they aren't recalculated when inputs haven't changed.

React.memo: Wraps a component. Prevents re-render if **props haven't changed**.

```
const MyComponent = React.memo(function({ name }) {
  return <div>{name}</div>;
});

// Won't re-render if 'name' prop hasn't changed
```

Controlled vs Uncontrolled Components

	Controlled	Uncontrolled
Data managed by	React state	DOM itself (via <code>ref</code>)
Source of truth	<code>useState</code>	DOM
Example	<code><input value={val} onChange={...}/></code>	<code><input ref={inputRef}/></code>

Keys in React

Keys help React identify which list items **changed, added, or removed**. Must be **unique among siblings**.

Why not use index as key?

When items reorder or get inserted/deleted, indexes shift → React maps wrong state to wrong element → bugs, incorrect UI, poor performance.

```
// Bad
items.map((item, i) => <Item key={i} />);

// Good
items.map(item => <Item key={item.id} />);
```

Higher Order Components (HOC)

A function that **takes a component** and **returns a new component** with added functionality. Pattern for reusing component logic.

```
function withAuth(WrappedComponent) {
  return function(props) {
    if (!isLoggedIn) return <Redirect to="/login" />;
    return <WrappedComponent {...props} />;
  };
}

const ProtectedDashboard = withAuth(Dashboard);
```

Section 4 — Browser & Network

What Happens When You Hit a URL in the Browser

1. **URL Parsing** → Browser identifies protocol, domain, path
 2. **DNS Resolution** → Domain name → IP address (checks cache first, then DNS servers)
 3. **TCP Connection** → 3-way handshake (SYN → SYN-ACK → ACK)
 4. **TLS Handshake** (HTTPS only) → Encrypted connection established
 5. **HTTP Request** → Browser sends `GET` request to server
 6. **Server Response** → Returns HTML, status code, headers
 7. **HTML Parsing** → Browser builds **DOM tree**
 8. **CSS Parsing** → Browser builds **CSSOM tree**
 9. **Render Tree** → DOM + CSSOM merged
 10. **Layout (Reflow)** → Calculate position/size of each element
 11. **Paint** → Pixels drawn to screen
 12. **JS Execution** → Scripts run (async/defer affects timing)
-

Section 5 — Key Interview Questions

setTimeout Output Order

```
setTimeout(() => console.log(1), 2000);  
setTimeout(() => console.log(2), 1000);  
setTimeout(() => console.log(3), 5000);  
  
// Output: 2 → 1 → 3 (sorted by delay, not by declaration order)
```

Event Loop Priority (Micro vs Macro)

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);
Promise.resolve().then(() => console.log("promise"));
console.log("end");

// Output: start → end → promise → timeout
// Microtasks (Promise) always run before Macrotasks (setTimeout)
```

Starvation

A situation where **microtasks keep queuing more microtasks**, preventing macrotasks (setTimeout callbacks) from ever executing — the call stack is never truly free.

Section 6 — Topic Reference Lists

DSA Topics to Cover

1. Sliding Window Techniques (Fixed & Variable)
2. Two Pointer Approach
3. Binary Search on Answer
4. Top K Elements using Heap
5. Graph Traversal (BFS & DFS)
6. Dynamic Programming (0/1 Knapsack, LIS)
7. Backtracking (Subsets, Permutations)
8. Trie-based Problems

Java Backend Topics

1. Multithreading and Synchronization
2. Java Streams API and Functional Programming
3. Exception Handling Best Practices
4. Spring Boot Annotations and Lifecycle
5. Microservices Architecture Fundamentals

6. Caching (Redis, In-Memory)
7. API Security (JWT, OAuth)
8. Database Indexing and Query Optimization

System Design Topics

1. Load Balancing Techniques
2. Database Sharding and Replication
3. Caching Strategies (CDN, Redis)
4. Event-Driven Architecture
5. Designing Scalable File Storage Systems
6. CAP Theorem and Trade-offs
7. Message Queues and Asynchronous Processing
8. High Availability and Fault Tolerance

SQL / Database Topics

- Normalisation & need of normalisation
- Schema building (VV-IMP)
- SQL Queries — rank, aggregate, window functions
- JOINS (inner, left, right, self, cross)
- GROUP BY vs PARTITION BY
- Window functions
- Multiple ways to solve same question — explain your reasoning